Responsive Quiz Application

COMSATS University Islamabad, Vehari Campus
Department of Computer Science

Course CSC336 Web Technologies **Class** BSSE **Term** Fall 2026
Instructor Yasmeen Jana **Marks** 10 **Submission date** 24 October 2026

Student name

Umar Sajjad

Registration number

GitHub repository URL

Project overview

QuizCraft is a responsive front-end quiz application for practicing HTML, CSS, JavaScript and Bootstrap. It presents randomized multiple-choice questions, checks answers immediately and provides a detailed result PDF. The default session contains five questions from a bank of thirty.

Assignment requirements

Required feature	Implementation
Client-side technologies	HTML, custom CSS, plain JavaScript and Bootstrap 5.3.8.
Randomized questions	5, 10 or 15 unique questions per attempt; shuffled answer choices.
Segmented progress	One segment per question with current, correct, incorrect and timeout states.
Countdown timer	15, 30 or 45 seconds per question; late answers score zero.
Dynamic highlighting	Green correct answer, red selected wrong answer and an explanation.
Dark and light modes	A theme toggle saves the preference in the same browser.
Download results	A local jsPDF export includes score, all answers and explanations.
Responsive layout	Bootstrap columns and media queries support small and large viewports.

Home Screen and Local Setup

The start screen lets a learner enter a name, choose the number of questions and set the time allowed for each question.

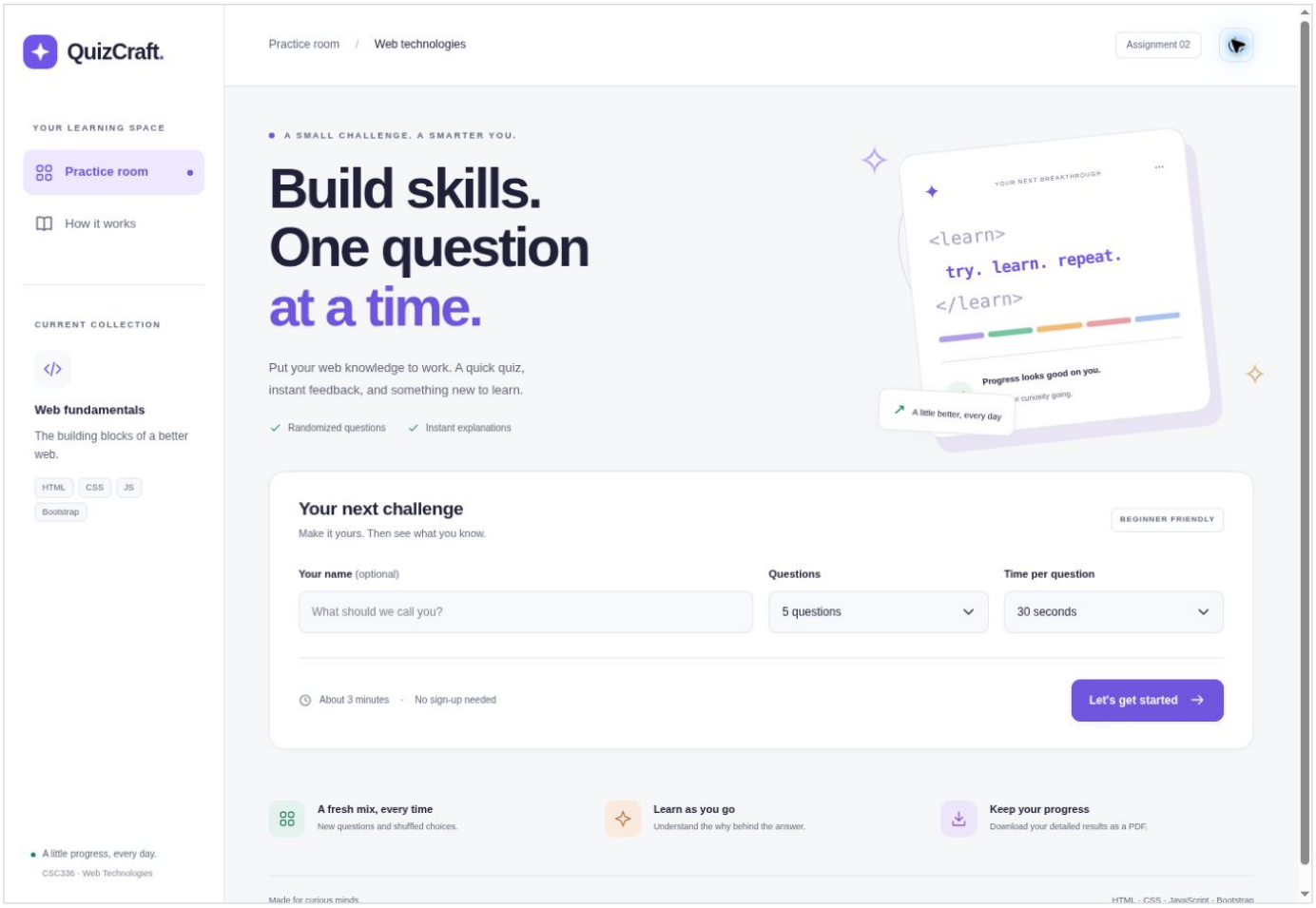


Figure 1 Actual desktop output in light mode, showing the setup form and start button.

Run in VS Code

1. Extract the project ZIP and open the QuizCraft folder in VS Code.
2. Open Terminal and run `npm run dev`. No npm install is needed.
3. Open `http://localhost:5173` and keep the terminal running.
4. Press Ctrl+C to stop. If PowerShell blocks npm.ps1, use `npm.cmd run dev`.

Node.js 18 or newer is needed for the development server. An alternative is to open `dist/index.html` directly in a current browser. All required CSS and JavaScript files are included locally.

Timed Questions and Answer Feedback

A question accepts one answer. After a selection, the choices lock and the correct option is highlighted. If the countdown expires, the application records a timeout, reveals the answer and waits for the learner to continue.

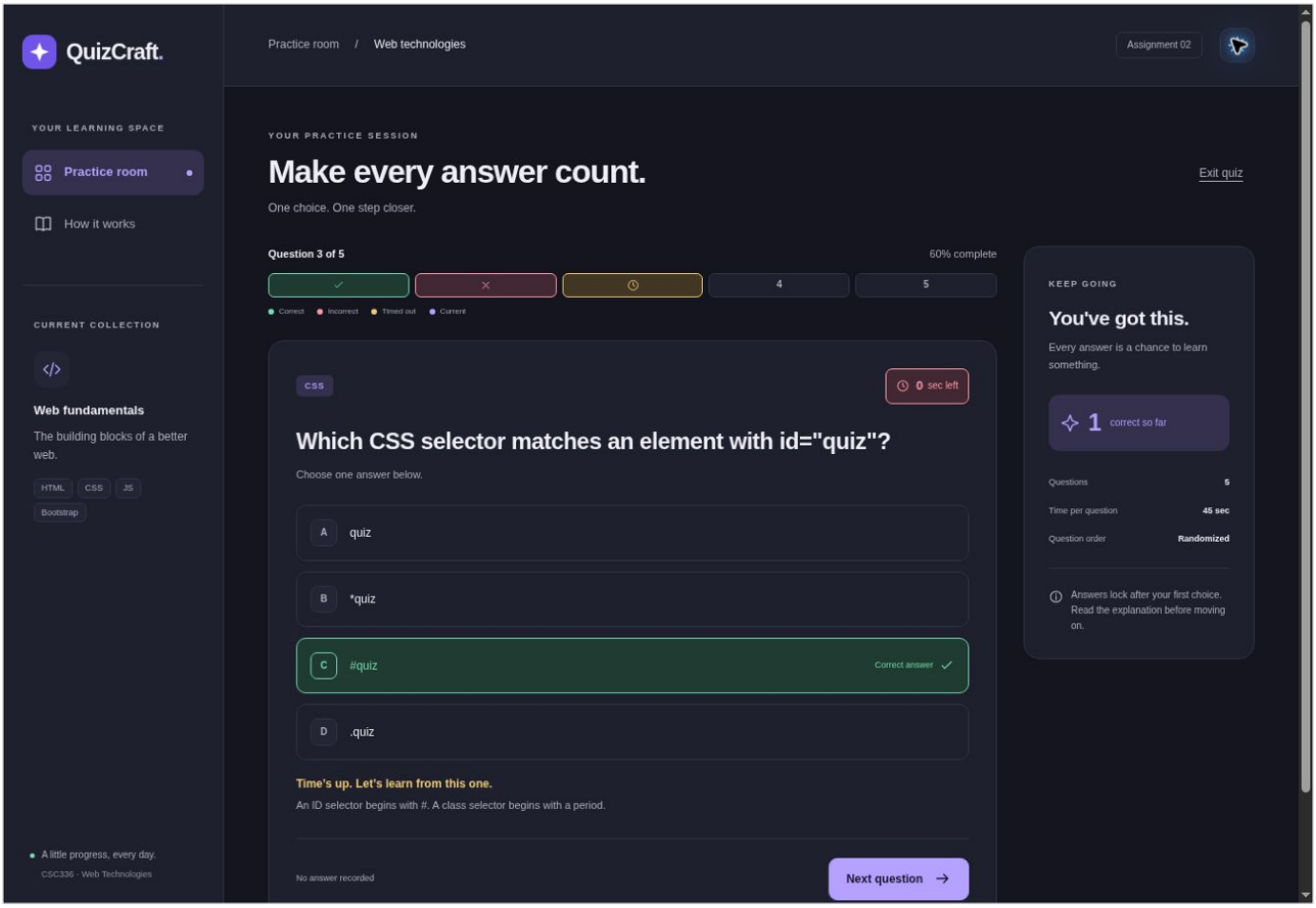


Figure 2 Actual dark-mode output. The segments show a correct answer, an incorrect answer and a timeout. The timer displays zero and the correct choice is revealed.

How the behavior is implemented

Randomization Fisher-Yates shuffling selects unique questions and changes the option order. Stable option IDs preserve the correct answer after shuffling.

Time validation Each question stores a deadline using performance.now(). An interval updates the display, while answer submission checks the deadline again. This prevents extra answering time after delayed callbacks or background-tab throttling.

Scoring A question can record only one answer. Correct answers earn one point; incorrect answers and timeouts earn zero. Percentage equals correct answers divided by total questions, multiplied by 100.

Results and PDF Export

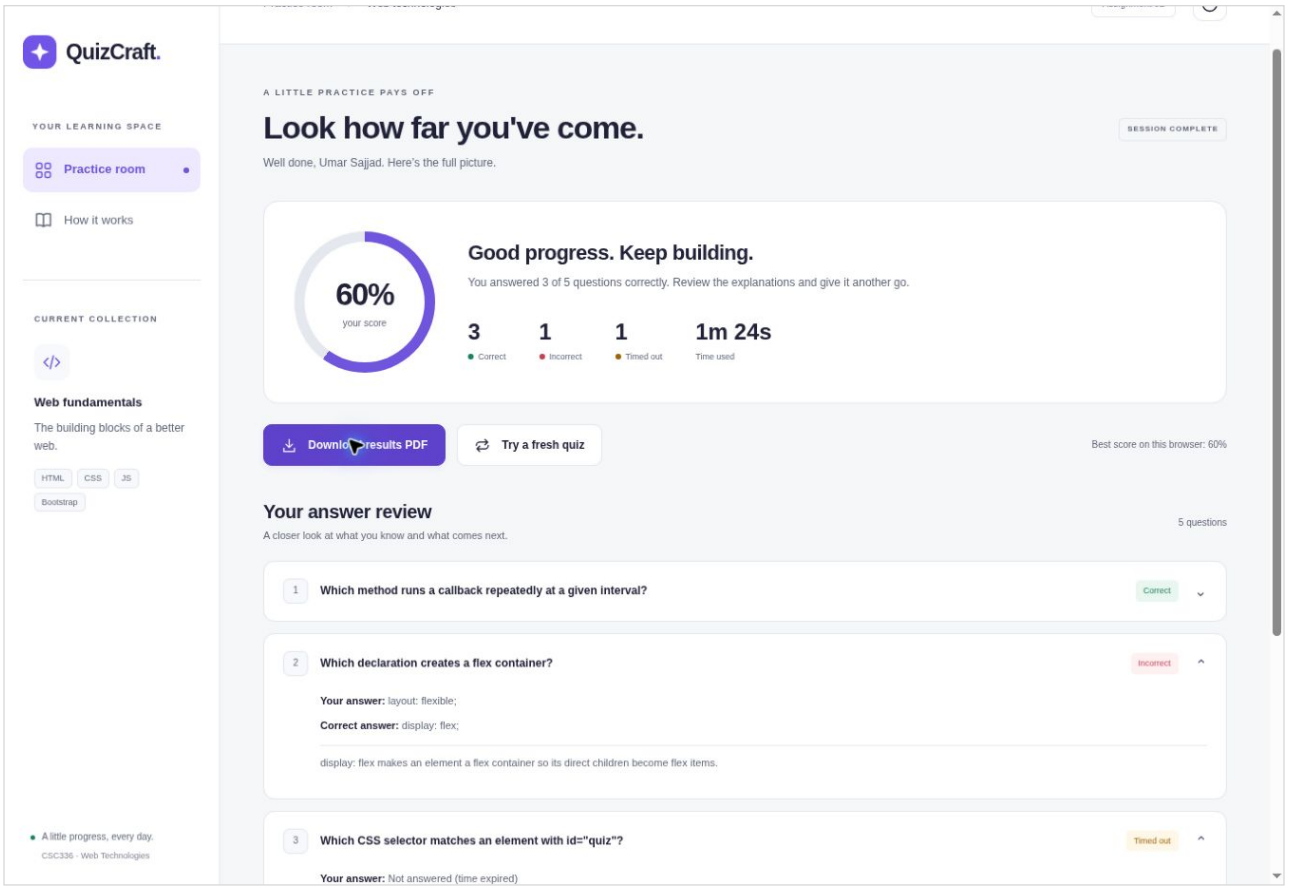


Figure 3 Actual completed session with three correct answers, one incorrect answer and one timeout. The score is 3 out of 5, or 60 percent.

Result information

The results screen shows the score, outcome totals and answering time. Expand each review item to see both answers and its explanation. A fresh quiz uses a new random sample.

Download results PDF creates a local PDF with the learner name, completion time, score and each question review. Longer exports continue on extra pages. The PDF includes its own fonts.

Verification

Check	Observed result
Automated quiz rules	8 tests passed, including deadline boundary and answer locking.
Browser session	Correct, incorrect and timed-out states produced the expected 60% score.
Theme preference	Dark mode survived a reload; light mode could be restored.
PDF generation	Valid 5-question and 15-question PDFs were generated and parsed.

Responsive Layout and Source Files

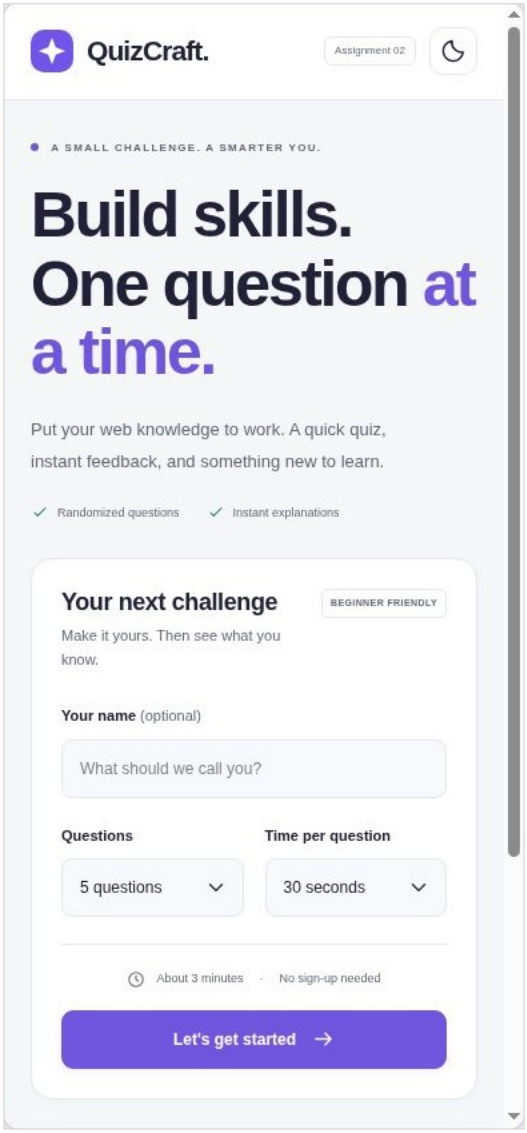


Figure 4 Actual home screen rendered in a 390 pixel iframe viewport. The sidebar is hidden and the form and start button stack vertically.

Responsive checks used real CSS viewport widths inside a browser frame. The question and feedback layouts had no horizontal overflow at 320, 390 or 768 pixels.

These checks verify the rendered layouts at those widths. Physical-device and cross-browser testing remain separate checks.

Source organization
index.html defines the screens.
styles.css defines layout and themes.
questions.js stores the question bank.
quiz-core.js checks quiz rules.
app.js updates the interface.
pdf-export.js creates results PDFs.

Bootstrap and jsPDF are included with their license notices. The development server uses only Node.js built-in modules.

Scope

This project demonstrates a front-end learning application. It does not include accounts, a database or secure examination controls. The browser stores the theme and best percentage; refreshing the page discards the current session.

References

Assignment 2 brief, CSC336 Web Technologies, COMSATS University Islamabad, Vehari Campus, Fall 2026.

Bootstrap documentation: getbootstrap.com/docs/5.3/getting-started/download/

jsPDF source and documentation: github.com/parallax/jsPDF

MDN Web Docs: Document.getElementById and Using media queries.